Case Study 5

Agentic Portfolio Intelligence

"Claude, prepare the IC meeting for NVIDIA."

Labs 9-10 — Weeks 10-11 — **MCP** + **LangGraph** + **CS1-CS4**

Course: Big Data and Intelligent Analytics
Term: Spring 2026

Designed by
Sri Krishnamurthy, CFA
sri@quantuniversity.com
Copyright © 2025 QuantUniversity LLC. All Rights Reserved.

Assigned: March 13, 2026
Due: March 27, 2026 (3:59 PM)

Version 4.0 — All technical review issues resolved

MANDATORY: CS1-CS4 Integration Required

This capstone case study requires working implementations from CS1, CS2, CS3, and CS4.
You will wrap your CS1-CS4 APIs as **MCP tools** and orchestrate them with **LangGraph agents**:

- **CS1:** Company metadata, portfolio management → MCP Resources
- **CS2:** Evidence collection (SEC, jobs, patents) → MCP Tools
- **CS3:** Scoring engine (V^R, H^R, Org-AI-R) → MCP Tools
- **CS4:** RAG search, score justifications → MCP Tools + Agent Memory

No mock data allowed. All agent tools must call your actual CS1-CS4 services.
Submissions using hardcoded data will receive a **maximum of 50% credit**.

"Show me the portfolio. Show me where to invest next."

This capstone transforms your CS1-CS4 platform into an **intelligent agent system**: **MCP Server** exposes your APIs as tools → **LangGraph Agents** orchestrate complex workflows

Real Data — Real Agents — Real PE Workflows

Weight	8% of final grade (100 base + 20 bonus points)
Format	Individual work
Prerequisite	**Working CS1, CS2, CS3, CS4 implementations**
Lab 9	MCP Server, Resources, Prompts, History Tracking, Dashboard
Lab 10	LangGraph Agents, Supervisor, HITL, Fund-AI-R, Metrics

Copyright © 2025 QuantUniversity LLC

Contents

1 Platform Architecture **4**
1.1 CS1-CS4 → MCP Tool Mapping [v4 FIX] 4
1.2 Required CS Client Interfaces [v4 FIX] 4

I Lab 9: MCP Server + Integration **6**

2 Lab 9 Preamble **6**
2.1 Learning Objectives 6
2.2 Tools Introduced 6

3 Task 9.1: Portfolio Data Service (8 pts) [REQUIRED] **7**

4 Task 9.2: MCP Server Core (12 pts) [MCP] **9**

5 Task 9.3: MCP Resources & Prompts (8 pts) [MCP] **13**

6 Task 9.4: Assessment History Tracking (6 pts) [NEW] **14**

7 Task 9.5: Evidence Display Component (6 pts) [NEW] **17**

8 Task 9.6: Portfolio Dashboard (10 pts) [REQUIRED] **20**

II Lab 10: LangGraph Agents + PE Workflows **22**

9 Lab 10 Preamble **22**
9.1 Learning Objectives [v4 FIX] 22

10 Task 10.1: LangGraph State Definitions (8 pts) [AGENT] **22**

11 Task 10.2: Specialist Agents (12 pts) [AGENT] **24**

12 Task 10.3: Supervisor with HITL (10 pts) [AGENT] **27**

13 Task 10.4: Agentic Due Diligence Workflow (10 pts) [AGENT] **29**

14 Task 10.5: Fund-AI-R Calculator (5 pts) [NEW] **30**

15 Task 10.6: Prometheus Metrics (5 pts) [NEW] **32**

16 Common Mistakes & Troubleshooting [v4 FIX] **34**

17 Testing Your Implementation [v4 FIX] **34**

18 Evaluation Criteria **36**

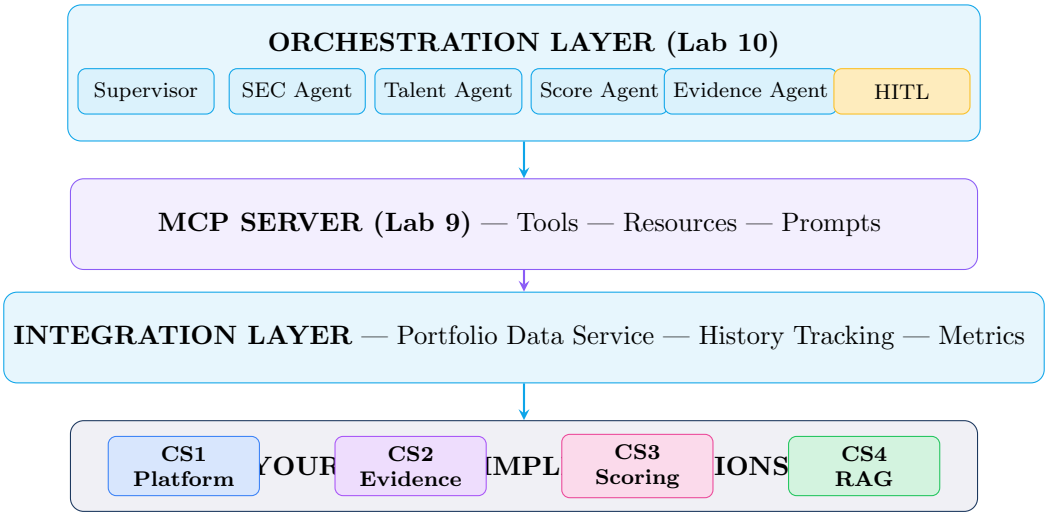
19 Submission **37**

Copyright © 2025 QuantUniversity LLC

1 Platform Architecture

Building on CS1-CS4

The Agentic Platform — Your CS1-CS4 Work Powers Intelligent Agents



1.1 CS1-CS4 → MCP Tool Mapping [v4 FIX]

MCP Tool	CS	API Endpoint	Client Method	Notes
calculate_org_air_score	CS3	GET /v2/assessments/{id}	cs3.get_assessment()	Returns CompanyAssessment
get_company_evidence	CS2	GET /v1/evidence/{id}	cs2.get_evidence()	Filters by dimension
generate_justification	CS4	POST /v1/justifications	cs4.generate_justification()	Requires RAG search
project_ebitda_impact	CS3	(Local calculation)	ebitda_calculator.project()	Uses v2.0 params
run_gap_analysis	CS3+CS4	POST /v1/gap-analysis	gap_analyzer.analyze()	Scoring + initiatives
get_portfolio_summary	CS1	GET /v1/portfolios/{id}	cs1.get_portfolio_companies()	Returns List[Company]

1.2 Required CS Client Interfaces [v4 FIX]

Your CS1-CS4 implementations must expose these interfaces for CS5 integration:

services/cs1_client.py — Required from CS1

```python
"""CS1 Client Interface - Students built this in Case Study 1."""
from dataclasses import dataclass
from enum import Enum
from typing import List, Optional

class Sector(Enum):
    TECHNOLOGY = "technology"
    HEALTHCARE = "healthcare"
    FINANCIAL_SERVICES = "financial_services"
    MANUFACTURING = "manufacturing"
    RETAIL = "retail"
    ENERGY = "energy"

@dataclass
class Company:
    company_id: str
    ticker: str
    name: str
    sector: Sector
    employee_count: int
```

```python
    revenue_mm: float
    portfolio_entry_date: Optional[str] = None

class CS1Client:
    """Client for CS1 Platform APIs."""

    def __init__(self, base_url: str = "http://localhost:8000"):
        self.base_url = base_url

    async def get_company(self, company_id: str) -> Company:
        """GET /v1/companies/{company_id}"""
        ...

    async def get_portfolio_companies(self, fund_id: str) -> List[Company]:
        """GET /v1/portfolios/{fund_id}/companies"""
        ...
```

services/cs3_client.py — Required from CS3

```python
"""CS3 Client Interface - Students built this in Case Study 3."""
from dataclasses import dataclass
from enum import Enum
from typing import Dict, Tuple

class Dimension(Enum):
    DATA_INFRASTRUCTURE = "data_infrastructure"
    AI_GOVERNANCE = "ai_governance"
    TECHNOLOGY_STACK = "technology_stack"
    TALENT = "talent"
    LEADERSHIP = "leadership"
    USE_CASE_PORTFOLIO = "use_case_portfolio"
    CULTURE = "culture"

@dataclass
class DimensionScore:
    dimension: Dimension
    score: float
    level: int
    evidence_count: int

@dataclass
class CompanyAssessment:
    company_id: str
    org_air_score: float
    vr_score: float
    hr_score: float
    synergy_score: float
    dimension_scores: Dict[Dimension, DimensionScore]
    confidence_interval: Tuple[float, float]
    evidence_count: int

class CS3Client:
    """Client for CS3 Scoring APIs."""

    async def get_assessment(self, company_id: str) -> CompanyAssessment:
        """GET /v2/assessments/{company_id}"""
        ...
```

Copyright © 2025 QuantUniversity LLC

Part I

Lab 9: MCP Server + Integration

2 Lab 9 Preamble

Date	March 13, 2026 (Thursday)
Focus	MCP Server, CS1-CS4 Integration, History Tracking, Dashboard
Time Estimate	Lecture: 2 hrs — Lab: 10 hrs — Challenge: +4 hrs

2.1 Learning Objectives

Learning Objectives

Bloom's Level	Objective
Remember	List the three MCP primitives: Tools, Resources, Prompts
Understand	Explain how MCP enables universal agent interoperability
Apply	Wrap CS1-CS4 APIs as MCP tools
Analyze	Compare MCP transport layers (stdio, SSE)
Evaluate	Assess tool schema design for agent usability
Create	Build a complete MCP server exposing your platform

2.2 Tools Introduced

Tools Introduced

Tool	Purpose	Why This Tool
MCP SDK	Model Context Protocol	Standard for agent-tool interoperability
Claude Desktop	MCP client	Test tools with Claude directly
Streamlit	Dashboard framework	Python-native, hot reload
nest_asyncio	Async compatibility	[v4 FIX]Required for Streamlit async
Your CS1-CS4	Data sources	REQUIRED — no mock data

Copyright © 2025 QuantUniversity LLC

3 Task 9.1: Portfolio Data Service (8 pts) [REQUIRED]

Building on CS1-CS4

This service is the **integration layer** between your CS1-CS4 APIs and the MCP server. It provides a unified interface for accessing all platform data.

services/integration/portfolio_data_service.py

```python
"""
Unified Portfolio Data Service - Integrates CS1, CS2, CS3, CS4.

This is the ONLY way to get data for agents and dashboards.
ALL data comes from YOUR CS1-CS4 implementations.
"""
from typing import List, Dict, Optional
from dataclasses import dataclass
from datetime import datetime
from decimal import Decimal
import structlog

# Import YOUR CS1-CS4 clients (these must exist from previous case studies)
from services.cs1_client import CS1Client, Company, Sector
from services.cs2_client import CS2Client, Evidence
from services.cs3_client import CS3Client, CompanyAssessment, Dimension
from services.cs4_client import CS4Client, ScoreJustification

logger = structlog.get_logger()

@dataclass
class PortfolioCompanyView:
    """Complete view of a portfolio company from CS1-CS4."""
    company_id: str
    ticker: str
    name: str
    sector: str
    org_air: float
    vr_score: float
    hr_score: float
    synergy_score: float
    dimension_scores: Dict[str, float]
    confidence_interval: tuple
    entry_org_air: float
    delta_since_entry: float
    evidence_count: int

class PortfolioDataService:
    """Unified data service integrating CS1-CS4."""

    def __init__(
        self,
        cs1_url: str = "http://localhost:8000",
        cs2_url: str = "http://localhost:8001",
        cs3_url: str = "http://localhost:8002",
    ):
        # Initialize all CS clients at construction time
        self.cs1 = CS1Client(base_url=cs1_url)
        self.cs2 = CS2Client(base_url=cs2_url)
        self.cs3 = CS3Client(base_url=cs3_url)
        self.cs4 = CS4Client()
        logger.info("portfolio_data_service_initialized")

    async def get_portfolio_view(
        self,
        fund_id: str,
    ) -> List[PortfolioCompanyView]:
        """Load portfolio from CS1, scores from CS3, evidence from CS2."""
```

```python
        # Call YOUR CS1 API
        companies = await self.cs1.get_portfolio_companies(fund_id)

        views = []
        for company in companies:
            # Call YOUR CS3 API
            assessment = await self.cs3.get_assessment(company.ticker)
            # Call YOUR CS2 API
            evidence = await self.cs2.get_evidence(company.ticker)

            entry_score = await self._get_entry_score(company.company_id)

            views.append(PortfolioCompanyView(
                company_id=company.company_id,
                ticker=company.ticker,
                name=company.name,
                sector=company.sector.value,
                org_air=assessment.org_air_score,
                vr_score=assessment.vr_score,
                hr_score=assessment.hr_score,
                synergy_score=assessment.synergy_score,
                dimension_scores={
                    d.value: s.score
                    for d, s in assessment.dimension_scores.items()
                },
                confidence_interval=assessment.confidence_interval,
                entry_org_air=entry_score,
                delta_since_entry=assessment.org_air_score - entry_score,
                evidence_count=len(evidence),
            ))

        return views

    async def _get_entry_score(self, company_id: str) -> float:
        """Get entry score from CS1 portfolio tracking."""
        # In production, query CS1's portfolio_positions table
        return 45.0  # Placeholder

# Singleton instance
portfolio_data_service = PortfolioDataService()
```

4 Task 9.2: MCP Server Core (12 pts) [MCP]

MCP Server:

Wrap CS1-CS4 as MCP Tools This is the key integration point. Your MCP server exposes CS1-CS4 capabilities as tools that **any LLM agent** can invoke.

mcp/server.py — MCP Server wrapping CS1-CS4 [v4 FIX]

```python
"""
PE Org-AI-R MCP Server - Universal agent interoperability.

This server exposes YOUR CS1-CS4 APIs as MCP tools.
Claude, GPT-4, or any MCP client can use your platform.

v4 FIX: All clients initialized at module level
"""
from typing import List, Dict, Any
from datetime import datetime  # v4 FIX: Import datetime
import json
from mcp.server import Server
from mcp.server.stdio import stdio_server
from mcp.types import (
    Tool, Resource, Prompt, PromptMessage,
    TextContent, ToolResult,
)
import structlog

# v4 FIX: Import ALL clients at module level (not inside functions)
from services.integration.portfolio_data_service import portfolio_data_service
from services.cs2_client import CS2Client
from services.cs3_client import CS3Client
from services.cs4_client import CS4Client
from services.value_creation.ebitda import ebitda_calculator
from services.value_creation.gap_analysis import gap_analyzer

logger = structlog.get_logger()

# v4 FIX: Initialize ALL clients at module level
mcp_server = Server("pe-orgair-server")
cs2_client = CS2Client()
cs3_client = CS3Client()
cs4_client = CS4Client()

# ============================================
# TOOLS - Executable functions
# ============================================
@mcp_server.list_tools()
async def list_tools() -> List[Tool]:
    """List all available tools."""
    return [
        Tool(
            name="calculate_org_air_score",
            description="""Calculate Org-AI-R score for a company using CS3 scoring engine.
Returns: Final score, V^R, H^R, synergy, confidence interval, dimension breakdown.""",
            inputSchema={
                "type": "object",
                "properties": {
                    "company_id": {"type": "string", "description": "Company ticker (e.g., 'NVDA')"},
                },
                "required": ["company_id"],
            },
        ),
        Tool(
            name="get_company_evidence",
            description="""Retrieve AI-readiness evidence for a company using CS2.
Returns: Evidence items with source type, content, confidence.""",
            inputSchema={
```

```
                "type": "object",
                "properties": {
                    "company_id": {"type": "string"},
                    "dimension": {
                        "type": "string",
                        "enum": ["data_infrastructure", "ai_governance",
                                "technology_stack", "talent", "leadership",
                                "use_case_portfolio", "culture", "all"],
                        "default": "all",
                    },
                    "limit": {"type": "integer", "default": 10},
                },
                "required": ["company_id"],
            },
        ),
        Tool(
            name="generate_justification",
            description="""Generate evidence-backed justification using CS4 RAG.
Returns: Score, level, rubric match, supporting evidence with citations.""",
            inputSchema={
                "type": "object",
                "properties": {
                    "company_id": {"type": "string"},
                    "dimension": {
                        "type": "string",
                        "enum": ["data_infrastructure", "ai_governance",
                                "technology_stack", "talent", "leadership",
                                "use_case_portfolio", "culture"],
                    },
                },
                "required": ["company_id", "dimension"],
            },
        ),
        Tool(
            name="project_ebitda_impact",
            description="""Project EBITDA impact from AI improvements using v2.0 model.
Returns: Conservative, base, optimistic scenarios with risk adjustment.""",
            inputSchema={
                "type": "object",
                "properties": {
                    "company_id": {"type": "string"},
                    "entry_score": {"type": "number", "minimum": 0, "maximum": 100},
                    "target_score": {"type": "number", "minimum": 0, "maximum": 100},
                    "h_r_score": {"type": "number", "minimum": 0, "maximum": 100},
                },
                "required": ["company_id", "entry_score", "target_score", "h_r_score"],
            },
        ),
        Tool(
            name="run_gap_analysis",
            description="""Analyze gaps and generate 100-day improvement plan.
Returns: Gap by dimension, priority ranking, initiatives, investment estimate.""",
            inputSchema={
                "type": "object",
                "properties": {
                    "company_id": {"type": "string"},
                    "target_org_air": {"type": "number", "minimum": 0, "maximum": 100},
                },
                "required": ["company_id", "target_org_air"],
            },
        ),
        Tool(
            name="get_portfolio_summary",
            description="""Get fund portfolio summary with Fund-AI-R.
Returns: Fund-AI-R, company breakdown, concentration metrics.""",
            inputSchema={
                "type": "object",
                "properties": {
                    "fund_id": {"type": "string"},
                },
```

```
                "required": ["fund_id"],
            },
        ),
    ]

@mcp_server.call_tool()
async def call_tool(name: str, arguments: dict) -> List[TextContent]:
    """Route tool calls to CS1-CS4 APIs."""
    logger.info("mcp_tool_call", tool=name, args=arguments)

    try:
        if name == "calculate_org_air_score":
            # Call YOUR CS3 scoring engine (module-level client)
            assessment = await cs3_client.get_assessment(arguments["company_id"])
            return [TextContent(text=json.dumps({
                "company_id": arguments["company_id"],
                "org_air": assessment.org_air_score,
                "vr_score": assessment.vr_score,
                "hr_score": assessment.hr_score,
                "synergy_score": assessment.synergy_score,
                "confidence_interval": list(assessment.confidence_interval),
                "dimension_scores": {
                    d.value: s.score
                    for d, s in assessment.dimension_scores.items()
                },
            }, indent=2))]

        elif name == "get_company_evidence":
            # v4 FIX: Use module-level client
            evidence = await cs2_client.get_evidence(
                company_id=arguments["company_id"],
                dimension=arguments.get("dimension", "all"),
                limit=arguments.get("limit", 10),
            )
            return [TextContent(text=json.dumps([
                {
                    "source_type": e.source_type.value,
                    "content": e.content[:500],
                    "confidence": e.confidence,
                    "signal_category": e.signal_category,
                }
                for e in evidence
            ], indent=2))]

        elif name == "generate_justification":
            from services.cs3_client import Dimension
            justification = await cs4_client.generate_justification(
                company_id=arguments["company_id"],
                dimension=Dimension(arguments["dimension"]),
            )
            return [TextContent(text=json.dumps({
                "dimension": arguments["dimension"],
                "score": justification.score,
                "level": justification.level,
                "level_name": justification.level_name,
                "evidence_strength": justification.evidence_strength,
                "rubric_criteria": justification.rubric_criteria,
                "supporting_evidence": [
                    {"source_type": e.source_type, "content": e.content[:300], "confidence": e.confidence}
                    for e in justification.supporting_evidence[:5]
                ],
                "gaps_identified": justification.gaps_identified,
            }, indent=2))]

        elif name == "project_ebitda_impact":
            projection = ebitda_calculator.project(
                company_id=arguments["company_id"],
                entry_score=arguments["entry_score"],
                exit_score=arguments["target_score"],
                h_r_score=arguments["h_r_score"],
```

```
            )
            return [TextContent(text=json.dumps({
                "delta_air": float(projection.delta_air),
                "scenarios": {
                    "conservative": f"{projection.conservative_pct:.2f}%",
                    "base": f"{projection.base_pct:.2f}%",
                    "optimistic": f"{projection.optimistic_pct:.2f}%",
                },
                "risk_adjusted": f"{projection.risk_adjusted_pct:.2f}%",
                "requires_approval": projection.requires_approval,
            }, indent=2))]

        elif name == "run_gap_analysis":
            assessment = await cs3_client.get_assessment(arguments["company_id"])
            current_scores = {d.value: s.score for d, s in assessment.dimension_scores.items()}
            analysis = gap_analyzer.analyze(
                company_id=arguments["company_id"],
                current_scores=current_scores,
                target_org_air=arguments["target_org_air"],
            )
            return [TextContent(text=json.dumps(analysis, indent=2))]

        elif name == "get_portfolio_summary":
            portfolio = await portfolio_data_service.get_portfolio_view(arguments["fund_id"])
            fund_air = sum(c.org_air for c in portfolio) / len(portfolio) if portfolio else 0
            return [TextContent(text=json.dumps({
                "fund_id": arguments["fund_id"],
                "fund_air": round(fund_air, 1),
                "company_count": len(portfolio),
                "companies": [{"ticker": c.ticker, "org_air": c.org_air, "sector": c.sector} for c in portfolio],
            }, indent=2))]

        else:
            return [TextContent(text=f"Unknown tool: {name}")]

    except Exception as e:
        logger.error("tool_error", tool=name, error=str(e))
        return [TextContent(text=f"Error: {str(e)}")]

# Main entry point
async def main():
    async with stdio_server() as (read_stream, write_stream):
        await mcp_server.run(read_stream, write_stream, mcp_server.create_initialization_options())

if __name__ == "__main__":
    import asyncio
    asyncio.run(main())
```

5 Task 9.3: MCP Resources & Prompts (8 pts) [MCP]

mcp/server.py (continued) — Resources & Prompts

```python
# ==========================================
# RESOURCES - Addressable data
# ==========================================
@mcp_server.list_resources()
async def list_resources() -> List[Resource]:
    return [
        Resource(
            uri="orgair://parameters/v2.0",
            name="Org-AI-R Scoring Parameters v2.0",
            description="Current scoring parameters: alpha, beta, gamma values",
        ),
        Resource(
            uri="orgair://sectors",
            name="Sector Definitions",
            description="Sector baselines and weights",
        ),
    ]

@mcp_server.read_resource()
async def read_resource(uri: str) -> str:
    if uri == "orgair://parameters/v2.0":
        return json.dumps({
            "version": "2.0",
            "alpha": 0.60, "beta": 0.12,
            "gamma_0": 0.0025, "gamma_1": 0.05, "gamma_2": 0.025, "gamma_3": 0.01,
        })
    elif uri == "orgair://sectors":
        return json.dumps({
            "technology": {"h_r_base": 85, "weight_talent": 0.18},
            "healthcare": {"h_r_base": 75, "weight_governance": 0.18},
        })
    return "{}"

# ==========================================
# PROMPTS - Reusable templates
# ==========================================
@mcp_server.list_prompts()
async def list_prompts() -> List[Prompt]:
    return [
        Prompt(
            name="due_diligence_assessment",
            description="Complete due diligence assessment for a company",
            arguments=[{"name": "company_id", "required": True}],
        ),
        Prompt(
            name="ic_meeting_prep",
            description="Prepare Investment Committee meeting package",
            arguments=[{"name": "company_id", "required": True}],
        ),
    ]

@mcp_server.get_prompt()
async def get_prompt(name: str, arguments: dict) -> List[PromptMessage]:
    if name == "due_diligence_assessment":
        return [PromptMessage(
            role="user",
            content=TextContent(text=f"""Perform due diligence for {arguments['company_id']}.
1. Calculate Org-AI-R score using calculate_org_air_score
2. For dimensions below 60, use generate_justification
3. Run gap_analysis with target_org_air=75
4. Project EBITDA impact"""),
        )]
    return []
```

6 Task 9.4: Assessment History Tracking (6 pts) [NEW]

Building on CS1-CS4

This task tracks assessment history over time, enabling trend analysis and delta calculations. Uses CS1 for storage and CS3 for current scores.

services/tracking/assessment_history.py [NEW]

```python
"""Assessment History Tracking - Stores score history for trend analysis."""
from dataclasses import dataclass
from datetime import datetime, timedelta
from typing import List, Dict, Optional
from decimal import Decimal
import structlog

from services.cs1_client import CS1Client
from services.cs3_client import CS3Client, CompanyAssessment

logger = structlog.get_logger()

@dataclass
class AssessmentSnapshot:
    """Single point-in-time assessment."""
    company_id: str
    timestamp: datetime
    org_air: Decimal
    vr_score: Decimal
    hr_score: Decimal
    synergy_score: Decimal
    dimension_scores: Dict[str, Decimal]
    confidence_interval: tuple
    evidence_count: int
    assessor_id: str
    assessment_type: str  # "screening", "limited", "full"

@dataclass
class AssessmentTrend:
    """Trend analysis for a company."""
    company_id: str
    current_org_air: float
    entry_org_air: float
    delta_since_entry: float
    delta_30d: Optional[float]
    delta_90d: Optional[float]
    trend_direction: str  # "improving", "stable", "declining"
    snapshot_count: int

class AssessmentHistoryService:
    """
    Tracks assessment history using CS1 for storage, CS3 for calculations.

    Students must implement:
    1. record_assessment() - Store new assessment snapshot
    2. get_history() - Retrieve snapshots for a company
    3. calculate_trend() - Compute trend metrics
    """

    def __init__(self, cs1_client: CS1Client, cs3_client: CS3Client):
        self.cs1 = cs1_client
        self.cs3 = cs3_client
        self._cache: Dict[str, List[AssessmentSnapshot]] = {}

    async def record_assessment(
        self,
        company_id: str,
        assessor_id: str,
```

```
        assessment_type: str = "full",
    ) -> AssessmentSnapshot:
        """
        Record current assessment as a snapshot.

        Flow:
        1. Call CS3 get_assessment() for current scores
        2. Create snapshot with timestamp
        3. Store in history (via CS1/Snowflake)
        4. Return snapshot
        """
        # Get current assessment from YOUR CS3
        assessment = await self.cs3.get_assessment(company_id)

        snapshot = AssessmentSnapshot(
            company_id=company_id,
            timestamp=datetime.utcnow(),
            org_air=Decimal(str(assessment.org_air_score)),
            vr_score=Decimal(str(assessment.vr_score)),
            hr_score=Decimal(str(assessment.hr_score)),
            synergy_score=Decimal(str(assessment.synergy_score)),
            dimension_scores={
                d.value: Decimal(str(s.score))
                for d, s in assessment.dimension_scores.items()
            },
            confidence_interval=assessment.confidence_interval,
            evidence_count=assessment.evidence_count,
            assessor_id=assessor_id,
            assessment_type=assessment_type,
        )

        # Store via CS1 (Snowflake)
        await self._store_snapshot(snapshot)

        # Update cache
        if company_id not in self._cache:
            self._cache[company_id] = []
        self._cache[company_id].append(snapshot)

        logger.info("assessment_recorded",
                    company_id=company_id,
                    org_air=float(snapshot.org_air))
        return snapshot

    async def _store_snapshot(self, snapshot: AssessmentSnapshot) -> None:
        """Store snapshot in Snowflake via CS1."""
        # In production: INSERT INTO assessment_history ...
        pass

    async def get_history(
        self,
        company_id: str,
        days: int = 365,
    ) -> List[AssessmentSnapshot]:
        """Retrieve assessment history from CS1/Snowflake."""
        # Check cache first
        if company_id in self._cache:
            cutoff = datetime.utcnow() - timedelta(days=days)
            return [s for s in self._cache[company_id] if s.timestamp >= cutoff]

        # Query CS1/Snowflake
        # SELECT * FROM assessment_history WHERE company_id = ? AND timestamp >= ?
        return []

    async def calculate_trend(self, company_id: str) -> AssessmentTrend:
        """Calculate trend metrics from history."""
        history = await self.get_history(company_id, days=365)

        if not history:
            # No history, get current assessment
```

```python
            current = await self.cs3.get_assessment(company_id)
            return AssessmentTrend(
                company_id=company_id,
                current_org_air=current.org_air_score,
                entry_org_air=current.org_air_score,
                delta_since_entry=0.0,
                delta_30d=None,
                delta_90d=None,
                trend_direction="stable",
                snapshot_count=0,
            )

        # Sort by timestamp
        history.sort(key=lambda s: s.timestamp)

        current = float(history[-1].org_air)
        entry = float(history[0].org_air)

        # Calculate deltas
        now = datetime.utcnow()
        delta_30d = None
        delta_90d = None

        for snapshot in reversed(history):
            age_days = (now - snapshot.timestamp).days
            if age_days >= 30 and delta_30d is None:
                delta_30d = current - float(snapshot.org_air)
            if age_days >= 90 and delta_90d is None:
                delta_90d = current - float(snapshot.org_air)
                break

        # Determine trend direction
        delta = current - entry
        if delta > 5:
            direction = "improving"
        elif delta < -5:
            direction = "declining"
        else:
            direction = "stable"

        return AssessmentTrend(
            company_id=company_id,
            current_org_air=current,
            entry_org_air=entry,
            delta_since_entry=round(delta, 1),
            delta_30d=round(delta_30d, 1) if delta_30d else None,
            delta_90d=round(delta_90d, 1) if delta_90d else None,
            trend_direction=direction,
            snapshot_count=len(history),
        )

# Factory function
def create_history_service(cs1: CS1Client, cs3: CS3Client) -> AssessmentHistoryService:
    return AssessmentHistoryService(cs1, cs3)
```

7 Task 9.5: Evidence Display Component (6 pts) [NEW]

Streamlit:

Visualize CS4 Justifications This Streamlit component displays evidence-backed justifications from CS4 RAG.

dashboard/components/evidence_display.py [NEW]

```python
"""Evidence Display Component - Streamlit component for CS4 justifications."""
import streamlit as st
from typing import Dict, List

from services.cs4_client import ScoreJustification

# Color coding by level
LEVEL_COLORS = {
    1: "#ef4444",  # red - Nascent
    2: "#f97316",  # orange - Developing
    3: "#eab308",  # yellow - Adequate
    4: "#22c55e",  # green - Good
    5: "#14b8a6",  # teal - Excellent
}

LEVEL_NAMES = {
    1: "Nascent", 2: "Developing", 3: "Adequate", 4: "Good", 5: "Excellent"
}

def render_evidence_card(justification: ScoreJustification) -> None:
    """
    Render a single dimension's evidence card.

    Shows:
    - Score badge with color coding (L1-L5)
    - Evidence list with source citations
    - Gaps identified section
    - Confidence indicator
    """
    color = LEVEL_COLORS.get(justification.level, "#6b7280")
    level_name = LEVEL_NAMES.get(justification.level, "Unknown")

    with st.container():
        # Header with score badge
        col1, col2, col3 = st.columns([4, 1, 1])

        with col1:
            st.markdown(f"### {justification.dimension.replace('_', ' ').title()}")

        with col2:
            st.markdown(
                f'<span style="background-color:{color};color:white;padding:4px 12px;'
                f'border-radius:12px;font-weight:bold;">L{justification.level}</span>',
                unsafe_allow_html=True
            )

        with col3:
            st.markdown(f"**{justification.score:.1f}**")

        # Evidence strength indicator
        strength_colors = {"strong": "#22c55e", "moderate": "#eab308", "weak": "#ef4444"}
        st.markdown(
            f"Evidence: <span style='color:{strength_colors.get(justification.evidence_strength, '#6b7280')}'>"
            f"**{justification.evidence_strength.title()}**</span>",
            unsafe_allow_html=True
        )

        # Rubric criteria
        st.info(f"**Rubric Match:** {justification.rubric_criteria}")
```

```python

        # Supporting evidence (from CS4 RAG)
        st.markdown("**Supporting Evidence:**")
        for i, evidence in enumerate(justification.supporting_evidence[:5], 1):
            with st.expander(f"[{evidence.source_type}] {evidence.content[:50]}...", expanded=False):
                st.write(evidence.content)
                st.caption(f"Confidence: {evidence.confidence:.0%}")
                if hasattr(evidence, 'source_url') and evidence.source_url:
                    st.markdown(f"[Source]({evidence.source_url})")

        # Gaps identified
        if justification.gaps_identified:
            st.warning("**Gaps Identified:**")
            for gap in justification.gaps_identified:
                st.markdown(f"- {gap}")

        st.divider()

def render_company_evidence_panel(
    company_id: str,
    justifications: Dict[str, ScoreJustification],
) -> None:
    """
    Render full evidence panel for a company.

    Displays:
    - Summary metrics
    - All 7 dimensions with evidence
    - Dimension filtering
    """
    st.header(f" Evidence Analysis: {company_id}")

    # Summary metrics
    total_evidence = sum(len(j.supporting_evidence) for j in justifications.values())
    avg_level = sum(j.level for j in justifications.values()) / len(justifications)
    strong_count = sum(1 for j in justifications.values() if j.evidence_strength == "strong")

    col1, col2, col3, col4 = st.columns(4)
    col1.metric("Total Evidence", total_evidence)
    col2.metric("Avg Level", f"L{avg_level:.1f}")
    col3.metric("Strong Evidence", f"{strong_count}/7")
    col4.metric("Dimensions", len(justifications))

    # Dimension tabs
    dim_names = [d.replace("_", " ").title() for d in justifications.keys()]
    tabs = st.tabs(dim_names)

    for tab, (dim, just) in zip(tabs, justifications.items()):
        with tab:
            render_evidence_card(just)

def render_evidence_summary_table(
    justifications: Dict[str, ScoreJustification],
) -> None:
    """Render compact summary table of all dimensions."""
    import pandas as pd

    data = []
    for dim, just in justifications.items():
        data.append({
            "Dimension": dim.replace("_", " ").title(),
            "Score": just.score,
            "Level": f"L{just.level}",
            "Evidence": just.evidence_strength.title(),
            "Items": len(just.supporting_evidence),
            "Gaps": len(just.gaps_identified),
        })

    df = pd.DataFrame(data)

```

```
    # Style the dataframe
    def color_level(val):
        level = int(val[1])
        return f"background-color: {LEVEL_COLORS.get(level, '#ffffff')}; color: white;"

    styled = df.style.applymap(color_level, subset=["Level"])
    st.dataframe(styled, use_container_width=True, hide_index=True)
```

8 Task 9.6: Portfolio Dashboard (10 pts) [REQUIRED]

dashboard/app.py — Portfolio Dashboard [v4 FIX]

```python
"""
PE Org-AI-R Portfolio Intelligence Dashboard.

ALL DATA comes from CS1-CS4 via PortfolioDataService.

v4 FIX: Added nest_asyncio for Streamlit async compatibility
"""
import streamlit as st
import plotly.express as px
import pandas as pd
import asyncio

# v4 FIX: Required for Streamlit async compatibility
import nest_asyncio
nest_asyncio.apply()

from services.integration.portfolio_data_service import portfolio_data_service
from services.tracking.assessment_history import create_history_service
from services.cs1_client import CS1Client
from services.cs3_client import CS3Client
from dashboard.components.evidence_display import render_evidence_summary_table

st.set_page_config(
    page_title="PE Org-AI-R Dashboard",
    page_icon="chart_with_upwards_trend",
    layout="wide",
)

# Sidebar
st.sidebar.title("PE Org-AI-R")
fund_id = st.sidebar.text_input("Fund ID", value="growth_fund_v")

# v4 FIX: Proper async pattern for Streamlit
@st.cache_data(ttl=300)
def load_portfolio(_fund_id: str):
    """Load portfolio data synchronously for Streamlit."""
    async def _load():
        return await portfolio_data_service.get_portfolio_view(_fund_id)

    loop = asyncio.new_event_loop()
    asyncio.set_event_loop(loop)
    try:
        portfolio = loop.run_until_complete(_load())
        return pd.DataFrame([
            {
                "ticker": c.ticker,
                "name": c.name,
                "sector": c.sector,
                "org_air": c.org_air,
                "vr_score": c.vr_score,
                "hr_score": c.hr_score,
                "delta": c.delta_since_entry,
                "evidence_count": c.evidence_count,
            }
            for c in portfolio
        ])
    finally:
        loop.close()

try:
    portfolio_df = load_portfolio(fund_id)
    st.sidebar.success(f"Loaded {len(portfolio_df)} companies from CS1-CS4")
except Exception as e:
    st.error(f"Failed to connect to CS1-CS4: {e}")
    st.info("Ensure CS1, CS2, CS3 services are running.")
    st.stop()
```

```python

# Main content
st.title("Portfolio Overview")

# Metrics row
fund_air = portfolio_df["org_air"].mean()
col1, col2, col3, col4 = st.columns(4)
with col1:
    st.metric("Fund-AI-R", f"{fund_air:.1f}")
with col2:
    st.metric("Companies", len(portfolio_df))
with col3:
    st.metric("Avg V^R", f"{portfolio_df['vr_score'].mean():.1f}")
with col4:
    avg_delta = portfolio_df['delta'].mean()
    st.metric("Avg Delta", f"{avg_delta:+.1f}")

# V^R vs H^R Scatter
fig = px.scatter(
    portfolio_df,
    x="vr_score", y="hr_score",
    size="org_air", color="sector",
    hover_name="name",
    title="Portfolio AI-Readiness Map (from CS3)",
    labels={"vr_score": "V^R (Idiosyncratic)", "hr_score": "H^R (Systematic)"},
)
fig.add_hline(y=60, line_dash="dash", line_color="gray", annotation_text="H^R Threshold")
fig.add_vline(x=60, line_dash="dash", line_color="gray", annotation_text="V^R Threshold")
st.plotly_chart(fig, use_container_width=True)

# Company table with conditional formatting
st.subheader("Portfolio Companies")
st.dataframe(
    portfolio_df.style.background_gradient(subset=["org_air"], cmap="RdYlGn"),
    use_container_width=True,
    hide_index=True,
)
```

Part II

Lab 10: LangGraph Agents + PE Workflows

9 Lab 10 Preamble

Date	March 20, 2026 (Thursday)
Focus	LangGraph Supervisor, Specialist Agents, HITL, Fund-AI-R, Metrics
Time Estimate	Lecture: 2 hrs — Lab: 10 hrs — Challenge: +4 hrs

9.1 Learning Objectives [v4 FIX]

Learning Objectives

Bloom's Level	Objective
Remember	List the four specialist agents and their responsibilities
Understand	Explain supervisor-worker pattern in multi-agent systems
Apply	Implement conditional routing based on workflow state
Analyze	Determine when HITL approval gates should trigger
Evaluate	Assess agent decisions for PE investment workflows
Create	Build end-to-end agentic due diligence pipeline

10 Task 10.1: LangGraph State Definitions (8 pts) [AGENT]

agents/state.py — Workflow State [v4 FIX]

```python
"""LangGraph state definitions for PE Org-AI-R agents."""
from typing import TypedDict, Annotated, List, Dict, Any, Optional, Literal
from datetime import datetime  # v4 FIX: Explicit import
import operator

class AgentMessage(TypedDict):
    """Message in agent conversation."""
    role: Literal["user", "assistant", "system", "tool"]
    content: str
    agent_name: Optional[str]
    timestamp: datetime

class DueDiligenceState(TypedDict):
    """State for due diligence workflow."""
    # Input
    company_id: str
    assessment_type: Literal["screening", "limited", "full"]
    requested_by: str

    # Messages (append-only via reducer)
    messages: Annotated[List[AgentMessage], operator.add]

    # Agent outputs
    sec_analysis: Optional[Dict[str, Any]]
    talent_analysis: Optional[Dict[str, Any]]
    scoring_result: Optional[Dict[str, Any]]
    evidence_justifications: Optional[Dict[str, Any]]
    value_creation_plan: Optional[Dict[str, Any]]

    # Workflow control
    next_agent: Optional[str]
    requires_approval: bool
```

```
    approval_reason: Optional[str]
    approval_status: Optional[Literal["pending", "approved", "rejected"]]
    approved_by: Optional[str]

    # Metadata
    started_at: datetime
    completed_at: Optional[datetime]
    total_tokens: int
    error: Optional[str]
```

11 Task 10.2: Specialist Agents (12 pts) [AGENT]

agents/specialists.py — Specialist Agents [v4 FIX]

```python
"""Specialist agents using MCP tools to access CS1-CS4."""
from typing import Dict, Any
from datetime import datetime  # v4 FIX: Import datetime
import json
import httpx
from langchain_openai import ChatOpenAI
from langchain_anthropic import ChatAnthropic
from langchain.tools import tool
import structlog

from agents.state import DueDiligenceState

logger = structlog.get_logger()

# v4 FIX: HTTP client for MCP tool calls (not non-existent MCPClient)
class MCPToolCaller:
    """HTTP client wrapper for MCP server tool calls."""

    def __init__(self, base_url: str = "http://localhost:3000"):
        self.base_url = base_url
        self.client = httpx.AsyncClient(timeout=30.0)

    async def call_tool(self, tool_name: str, arguments: dict) -> str:
        """Call MCP tool via HTTP."""
        response = await self.client.post(
            f"{self.base_url}/tools/{tool_name}",
            json=arguments,
        )
        response.raise_for_status()
        return response.json().get("result", "{}")

mcp_client = MCPToolCaller()

# ============================================
# TOOLS (wrap MCP calls)
# ============================================
@tool
async def get_org_air_score(company_id: str) -> str:
    """Get Org-AI-R score for a company via MCP."""
    return await mcp_client.call_tool("calculate_org_air_score", {"company_id": company_id})

@tool
async def get_evidence(company_id: str, dimension: str = "all") -> str:
    """Get evidence for a company via MCP."""
    return await mcp_client.call_tool("get_company_evidence", {"company_id": company_id, "dimension": dimension})

@tool
async def get_justification(company_id: str, dimension: str) -> str:
    """Get CS4 justification for a dimension via MCP."""
    return await mcp_client.call_tool("generate_justification", {"company_id": company_id, "dimension": dimension
    ↪ })

@tool
async def get_gap_analysis(company_id: str, target: float) -> str:
    """Run gap analysis via MCP."""
    return await mcp_client.call_tool("run_gap_analysis", {"company_id": company_id, "target_org_air": target})

# ============================================
# SPECIALIST AGENTS
# ============================================
class SECAnalysisAgent:
    """Agent specialized in SEC filing analysis."""

    def __init__(self):
        self.llm = ChatOpenAI(model="gpt-4o", temperature=0.3)
        self.tools = [get_evidence]
```

```python

    async def analyze(self, state: DueDiligenceState) -> Dict[str, Any]:
        company_id = state["company_id"]

        # Get evidence via MCP tool
        evidence_result = await get_evidence.ainvoke({"company_id": company_id, "dimension": "all"})

        return {
            "sec_analysis": {
                "company_id": company_id,
                "findings": json.loads(evidence_result) if evidence_result else [],
                "dimensions_covered": ["data_infrastructure", "ai_governance", "technology_stack"],
            },
            "messages": [{
                "role": "assistant",
                "content": f"SEC analysis complete for {company_id}",
                "agent_name": "sec_analyst",
                "timestamp": datetime.utcnow(),  # v4 FIX: Now works
            }],
        }

class ScoringAgent:
    """Agent for calculating and explaining Org-AI-R scores."""

    def __init__(self):
        self.llm = ChatAnthropic(model="claude-sonnet-4-20250514", temperature=0.2)
        self.tools = [get_org_air_score, get_justification]

    async def calculate(self, state: DueDiligenceState) -> Dict[str, Any]:
        company_id = state["company_id"]

        # Get score via MCP tool
        score_result = await get_org_air_score.ainvoke({"company_id": company_id})
        score_data = json.loads(score_result)

        # Check HITL requirement
        org_air = score_data["org_air"]
        requires_approval = org_air > 85 or org_air < 40
        approval_reason = f"Score {org_air:.1f} outside normal range [40, 85]" if requires_approval else None

        return {
            "scoring_result": score_data,
            "requires_approval": requires_approval,
            "approval_reason": approval_reason,
            "approval_status": "pending" if requires_approval else None,
            "messages": [{
                "role": "assistant",
                "content": f"Scoring complete: Org-AI-R = {org_air:.1f}" + (" [REQUIRES APPROVAL]" if requires_approval else ""),
                "agent_name": "scorer",
                "timestamp": datetime.utcnow(),
            }],
        }

class EvidenceAgent:
    """Agent for evidence retrieval and justification."""

    def __init__(self):
        self.llm = ChatOpenAI(model="gpt-4o", temperature=0.3)
        self.tools = [get_justification]

    async def justify(self, state: DueDiligenceState) -> Dict[str, Any]:
        company_id = state["company_id"]

        dimensions = ["data_infrastructure", "talent", "use_case_portfolio"]
        justifications = {}

        for dim in dimensions:
            result = await get_justification.ainvoke({"company_id": company_id, "dimension": dim})
            justifications[dim] = json.loads(result)
```

```python

        return {
            "evidence_justifications": {"company_id": company_id, "justifications": justifications},
            "messages": [{
                "role": "assistant",
                "content": f"Generated justifications for {len(justifications)} dimensions",
                "agent_name": "evidence_agent",
                "timestamp": datetime.utcnow(),
            }],
        }

class ValueCreationAgent:
    """Agent for EBITDA projections and value creation planning."""

    def __init__(self):
        self.llm = ChatOpenAI(model="gpt-4o", temperature=0.3)
        self.tools = [get_gap_analysis]

    async def plan(self, state: DueDiligenceState) -> Dict[str, Any]:
        company_id = state["company_id"]

        gap_result = await get_gap_analysis.ainvoke({"company_id": company_id, "target": 80.0})
        gap_data = json.loads(gap_result)

        # Check HITL for large projections (simplified)
        projected_impact = gap_data.get("projected_ebitda_pct", 3.5)
        requires_approval = projected_impact > 5.0 or state.get("requires_approval", False)

        return {
            "value_creation_plan": {"company_id": company_id, "gap_analysis": gap_data},
            "requires_approval": requires_approval,
            "approval_reason": state.get("approval_reason") or (f"EBITDA {projected_impact}% > 5%" if projected_impact > 5 else None),
            "messages": [{
                "role": "assistant",
                "content": f"Value creation plan complete. Projected impact: {projected_impact}%",
                "agent_name": "value_creator",
                "timestamp": datetime.utcnow(),
            }],
        }

# Instantiate agents
sec_agent = SECAnalysisAgent()
scoring_agent = ScoringAgent()
evidence_agent = EvidenceAgent()
value_agent = ValueCreationAgent()
```

12 Task 10.3: Supervisor with HITL (10 pts) [AGENT]

HITL Approval:

Human-in-the-Loop Approval The supervisor includes **HITL approval gates** for:

- Scores outside normal range (below 40 or above 85)
- EBITDA projections exceeding 5%
- Any assessment flagged for review

agents/supervisor.py — LangGraph Supervisor

```python
"""Supervisor agent with HITL approval gates."""
from typing import Literal, Dict, Any
from datetime import datetime
from langgraph.graph import StateGraph, END
from langgraph.checkpoint.memory import MemorySaver
import structlog

from agents.state import DueDiligenceState
from agents.specialists import sec_agent, scoring_agent, evidence_agent, value_agent

logger = structlog.get_logger()

async def supervisor_node(state: DueDiligenceState) -> Dict[str, Any]:
    """Supervisor decides which agent to run next."""
    if state.get("requires_approval") and state.get("approval_status") == "pending":
        return {"next_agent": "hitl_approval"}

    if not state.get("sec_analysis"):
        return {"next_agent": "sec_analyst"}
    elif not state.get("scoring_result"):
        return {"next_agent": "scorer"}
    elif not state.get("evidence_justifications"):
        return {"next_agent": "evidence_agent"}
    elif not state.get("value_creation_plan") and state["assessment_type"] != "screening":
        return {"next_agent": "value_creator"}
    else:
        return {"next_agent": "complete"}

async def sec_analyst_node(state: DueDiligenceState) -> Dict[str, Any]:
    return await sec_agent.analyze(state)

async def scorer_node(state: DueDiligenceState) -> Dict[str, Any]:
    return await scoring_agent.calculate(state)

async def evidence_node(state: DueDiligenceState) -> Dict[str, Any]:
    return await evidence_agent.justify(state)

async def value_creator_node(state: DueDiligenceState) -> Dict[str, Any]:
    return await value_agent.plan(state)

async def hitl_approval_node(state: DueDiligenceState) -> Dict[str, Any]:
    """Human-in-the-loop approval gate."""
    logger.warning("hitl_approval_required", company_id=state["company_id"], reason=state.get("approval_reason"))

    # In production: Send Slack/email notification, wait for response
    # For exercise: Auto-approve after logging
    return {
        "approval_status": "approved",
        "approved_by": "exercise_auto_approve",
        "messages": [{
            "role": "system",
            "content": f"HITL approval granted: {state.get('approval_reason')}",
            "agent_name": "hitl",
            "timestamp": datetime.utcnow(),
        }],
```

```
    }

async def complete_node(state: DueDiligenceState) -> Dict[str, Any]:
    return {
        "completed_at": datetime.utcnow(),
        "messages": [{"role": "assistant", "content": f"Due diligence complete for {state['company_id']}", "
    ↪ agent_name": "supervisor", "timestamp": datetime.utcnow()}],
    }

def create_due_diligence_graph():
    workflow = StateGraph(DueDiligenceState)

    workflow.add_node("supervisor", supervisor_node)
    workflow.add_node("sec_analyst", sec_analyst_node)
    workflow.add_node("scorer", scorer_node)
    workflow.add_node("evidence_agent", evidence_node)
    workflow.add_node("value_creator", value_creator_node)
    workflow.add_node("hitl_approval", hitl_approval_node)
    workflow.add_node("complete", complete_node)

    workflow.add_conditional_edges("supervisor", lambda s: s["next_agent"], {
        "sec_analyst": "sec_analyst", "scorer": "scorer", "evidence_agent": "evidence_agent",
        "value_creator": "value_creator", "hitl_approval": "hitl_approval", "complete": "complete",
    })

    for agent in ["sec_analyst", "scorer", "evidence_agent", "value_creator"]:
        workflow.add_edge(agent, "supervisor")
    workflow.add_edge("hitl_approval", "supervisor")
    workflow.add_edge("complete", END)
    workflow.set_entry_point("supervisor")

    return workflow.compile(checkpointer=MemorySaver())

dd_graph = create_due_diligence_graph()
```

13 Task 10.4: Agentic Due Diligence Workflow (10 pts) [AGENT]

Practical Exercise:

Complete Multi-Agent Workflow This exercise runs the complete agentic due diligence process.

exercises/agentic_due_diligence.py

```python
"""Agentic Due Diligence - Complete Multi-Agent Workflow."""
import asyncio
from datetime import datetime
from agents.supervisor import dd_graph
from agents.state import DueDiligenceState

async def run_due_diligence(company_id: str, assessment_type: str = "full") -> DueDiligenceState:
    initial_state: DueDiligenceState = {
        "company_id": company_id, "assessment_type": assessment_type, "requested_by": "analyst",
        "messages": [], "sec_analysis": None, "talent_analysis": None, "scoring_result": None,
        "evidence_justifications": None, "value_creation_plan": None, "next_agent": None,
        "requires_approval": False, "approval_reason": None, "approval_status": None,
        "approved_by": None, "started_at": datetime.utcnow(), "completed_at": None,
        "total_tokens": 0, "error": None,
    }

    config = {"configurable": {"thread_id": f"dd-{company_id}-{datetime.now().isoformat()}"}}
    return await dd_graph.ainvoke(initial_state, config)

async def main():
    print("=" * 60)
    print("PE Org-AI-R: Agentic Due Diligence")
    print("=" * 60)

    result = await run_due_diligence("NVDA", "full")

    print(f"\nOrg-AI-R: {result['scoring_result']['org_air']:.1f}")
    print(f"HITL Required: {result.get('requires_approval', False)}")
    print(f"Status: {result.get('approval_status', 'N/A')}")
    print("\nAll data came from CS1-CS4 via MCP tools.")

if __name__ == "__main__":
    asyncio.run(main())
```

14 Task 10.5: Fund-AI-R Calculator (5 pts) [NEW]

Building on CS1-CS4

The Fund-AI-R Calculator aggregates company-level Org-AI-R scores into a portfolio-level metric using EV-weighted averaging.

services/analytics/fund_air.py [NEW]

```python
"""Fund-AI-R Calculator - Portfolio-level AI-readiness aggregation."""
from dataclasses import dataclass
from typing import List, Dict
import structlog

from services.integration.portfolio_data_service import PortfolioCompanyView

logger = structlog.get_logger()

# Sector benchmarks for quartile calculation
SECTOR_BENCHMARKS = {
    "technology": {"q1": 75, "q2": 65, "q3": 55, "q4": 45},
    "healthcare": {"q1": 70, "q2": 58, "q3": 48, "q4": 38},
    "financial_services": {"q1": 72, "q2": 60, "q3": 50, "q4": 40},
    "manufacturing": {"q1": 68, "q2": 55, "q3": 45, "q4": 35},
    "retail": {"q1": 65, "q2": 52, "q3": 42, "q4": 32},
    "energy": {"q1": 60, "q2": 48, "q3": 38, "q4": 28},
}

@dataclass
class FundMetrics:
    """Fund-level AI-readiness metrics."""
    fund_id: str
    fund_air: float  # EV-weighted average Org-AI-R
    company_count: int
    quartile_distribution: Dict[int, int]
    sector_hhi: float  # Herfindahl-Hirschman Index
    avg_delta_since_entry: float
    total_ev_mm: float
    ai_leaders_count: int  # Score >= 70
    ai_laggards_count: int  # Score < 50

class FundAIRCalculator:
    """Calculates Fund-AI-R using EV-weighted aggregation."""

    def calculate_fund_metrics(
        self,
        fund_id: str,
        companies: List[PortfolioCompanyView],
        enterprise_values: Dict[str, float],
    ) -> FundMetrics:
        if not companies:
            raise ValueError("Cannot calculate Fund-AI-R for empty portfolio")

        # EV-weighted Org-AI-R
        total_ev = sum(enterprise_values.get(c.company_id, 100.0) for c in companies)
        weighted_sum = sum(enterprise_values.get(c.company_id, 100.0) * c.org_air for c in companies)
        fund_air = weighted_sum / total_ev if total_ev > 0 else 0.0

        # Quartile distribution
        quartile_dist = {1: 0, 2: 0, 3: 0, 4: 0}
        for c in companies:
            q = self._get_quartile(c.org_air, c.sector)
            quartile_dist[q] += 1

        # Sector HHI
        sector_ev: Dict[str, float] = {}
        for c in companies:
```

```
            ev = enterprise_values.get(c.company_id, 100.0)
            sector_ev[c.sector] = sector_ev.get(c.sector, 0.0) + ev
        hhi = sum((ev / total_ev) ** 2 for ev in sector_ev.values()) if total_ev > 0 else 0

        return FundMetrics(
            fund_id=fund_id,
            fund_air=round(fund_air, 1),
            company_count=len(companies),
            quartile_distribution=quartile_dist,
            sector_hhi=round(hhi, 4),
            avg_delta_since_entry=round(sum(c.delta_since_entry for c in companies) / len(companies), 1),
            total_ev_mm=round(total_ev, 1),
            ai_leaders_count=sum(1 for c in companies if c.org_air >= 70),
            ai_laggards_count=sum(1 for c in companies if c.org_air < 50),
        )

    def _get_quartile(self, score: float, sector: str) -> int:
        benchmarks = SECTOR_BENCHMARKS.get(sector, SECTOR_BENCHMARKS["technology"])
        if score >= benchmarks["q1"]: return 1
        elif score >= benchmarks["q2"]: return 2
        elif score >= benchmarks["q3"]: return 3
        else: return 4

fund_air_calculator = FundAIRCalculator()
```

15 Task 10.6: Prometheus Metrics (5 pts) [NEW]

services/observability/metrics.py [NEW]

```python
"""Prometheus Metrics for MCP Server and LangGraph Agents."""
from prometheus_client import Counter, Histogram, Gauge
from functools import wraps
import time

# MCP SERVER METRICS
MCP_TOOL_CALLS = Counter(
    "mcp_tool_calls_total", "Total MCP tool invocations", ["tool_name", "status"]
)
MCP_TOOL_DURATION = Histogram(
    "mcp_tool_duration_seconds", "MCP tool execution duration", ["tool_name"],
    buckets=[0.1, 0.25, 0.5, 1.0, 2.5, 5.0, 10.0]
)

# LANGGRAPH AGENT METRICS
AGENT_INVOCATIONS = Counter(
    "agent_invocations_total", "Total agent invocations", ["agent_name", "status"]
)
AGENT_DURATION = Histogram(
    "agent_duration_seconds", "Agent execution duration", ["agent_name"],
    buckets=[0.5, 1.0, 2.5, 5.0, 10.0, 30.0]
)

# HITL METRICS
HITL_APPROVALS = Counter(
    "hitl_approvals_total", "HITL approval requests", ["reason", "decision"]
)

# CS1-CS4 INTEGRATION METRICS
CS_CLIENT_CALLS = Counter(
    "cs_client_calls_total", "Calls to CS1-CS4 services", ["service", "endpoint", "status"]
)

# DECORATORS
def track_mcp_tool(tool_name: str):
    """Decorator to track MCP tool metrics."""
    def decorator(func):
        @wraps(func)
        async def wrapper(*args, **kwargs):
            start = time.perf_counter()
            try:
                result = await func(*args, **kwargs)
                MCP_TOOL_CALLS.labels(tool_name=tool_name, status="success").inc()
                return result
            except Exception as e:
                MCP_TOOL_CALLS.labels(tool_name=tool_name, status="error").inc()
                raise
            finally:
                MCP_TOOL_DURATION.labels(tool_name=tool_name).observe(time.perf_counter() - start)
        return wrapper
    return decorator

def track_agent(agent_name: str):
    """Decorator to track agent metrics."""
    def decorator(func):
        @wraps(func)
        async def wrapper(*args, **kwargs):
            start = time.perf_counter()
            try:
                result = await func(*args, **kwargs)
                AGENT_INVOCATIONS.labels(agent_name=agent_name, status="success").inc()
                return result
            except Exception as e:
                AGENT_INVOCATIONS.labels(agent_name=agent_name, status="error").inc()
                raise
            finally:
```

```
                AGENT_DURATION.labels(agent_name=agent_name).observe(time.perf_counter() - start)
        return wrapper
    return decorator

def track_cs_client(service: str, endpoint: str):
    """Decorator to track CS client calls."""
    def decorator(func):
        @wraps(func)
        async def wrapper(*args, **kwargs):
            try:
                result = await func(*args, **kwargs)
                CS_CLIENT_CALLS.labels(service=service, endpoint=endpoint, status="success").inc()
                return result
            except Exception as e:
                CS_CLIENT_CALLS.labels(service=service, endpoint=endpoint, status="error").inc()
                raise
        return wrapper
    return decorator
```

16 Common Mistakes & Troubleshooting [v4 FIX]

Common Mistakes

Top 5 Student Mistakes in CS5:

1. **Hardcoding mock data** instead of calling CS1-CS4 APIs
   - *Symptom:* Dashboard works but shows fake/static data
   - *Fix:* Verify MCP tools call your actual CS client classes
   - *Test:* Stop CS3 service — your dashboard should error, not show data
2. **Forgetting async/await** in LangGraph nodes
   - *Symptom:* `RuntimeWarning: coroutine was never awaited`
   - *Fix:* All MCP tool calls must use `await`
3. **Missing datetime import**
   - *Symptom:* `NameError: name 'datetime' is not defined`
   - *Fix:* Add `from datetime import datetime` at top of file
4. **Streamlit async incompatibility**
   - *Symptom:* `RuntimeError: This event loop is already running`
   - *Fix:* Add `import nest_asyncio; nest_asyncio.apply()` at start
5. **Missing HITL trigger for high scores**
   - *Symptom:* Scores of 95 approved without human review
   - *Fix:* Check BOTH bounds: `org_air > 85 or org_air < 40`

17 Testing Your Implementation [v4 FIX]

Testing:

Verify CS1-CS4 Integration These tests verify your MCP tools actually call CS1-CS4 (not hard-coded data).

tests/test_mcp_integration.py [NEW]

```python
"""Tests to verify MCP tools call CS1-CS4."""
import pytest
from unittest.mock import AsyncMock, patch

@pytest.mark.asyncio
async def test_calculate_org_air_calls_cs3():
    """Verify calculate_org_air_score calls CS3 client."""
    from mcp.server import call_tool, cs3_client

    # Mock the CS3 client
    with patch.object(cs3_client, 'get_assessment', new_callable=AsyncMock) as mock:
        mock.return_value = MockAssessment(org_air_score=72.5)

        result = await call_tool("calculate_org_air_score", {"company_id": "NVDA"})

        # CRITICAL: Verify CS3 was called
        mock.assert_called_once_with("NVDA")

@pytest.mark.asyncio
async def test_no_hardcoded_data():
    """Fail if tools return data without calling CS clients."""
    from mcp.server import call_tool, cs3_client

    with patch.object(cs3_client, 'get_assessment', new_callable=AsyncMock) as mock:
        mock.side_effect = ConnectionError("CS3 not running")

```

Copyright © 2025 QuantUniversity LLC

```
        # Should raise error, NOT return hardcoded data
        with pytest.raises(Exception):
            await call_tool("calculate_org_air_score", {"company_id": "NVDA"})
```

Run tests: `pytest tests/ -v --tb=short`

18 Evaluation Criteria

Requirements

Core Requirements (100 points)

Lab 9: MCP Server + Integration (50 points)

- ✓ Task 9.1: Portfolio Data Service (CS1-CS4 integration) (8 pts)
- ✓ Task 9.2: MCP Server Core (6 tools minimum) (12 pts)
- ✓ Task 9.3: MCP Resources & Prompts (8 pts)
- ✓ Task 9.4: Assessment History Tracking (6 pts) **[NEW]**
- ✓ Task 9.5: Evidence Display Component (6 pts) **[NEW]**
- ✓ Task 9.6: Portfolio Dashboard (10 pts)

Lab 10: LangGraph Agents + Workflows (50 points)

- ✓ Task 10.1: LangGraph State Definitions (8 pts)
- ✓ Task 10.2: Specialist Agents (4 agents minimum) (12 pts)
- ✓ Task 10.3: Supervisor with HITL (10 pts)
- ✓ Task 10.4: Agentic Due Diligence Workflow (10 pts)
- ✓ Task 10.5: Fund-AI-R Calculator (5 pts) **[NEW]**
- ✓ Task 10.6: Prometheus Metrics (5 pts) **[NEW]**

Extensions (+20 bonus points)

- ✓ Mem0 Semantic Memory (+5 pts)
- ✓ Investment Tracker with ROI (+5 pts)
- ✓ IC Memo Generator (Word doc) (+5 pts)
- ✓ LP Letter Generator (+5 pts)

Warning

Grading Penalty for Mock Data

Submissions using hardcoded data will receive **maximum 50% credit**.

We verify by:

- Checking MCP tools call CS1-CS4 clients
- Running pytest with CS services stopped (should error)
- Reviewing code for hardcoded return values

Copyright © 2025 QuantUniversity LLC

19 Submission

Due: **March 27, 2026 at 3:59 PM**

```
cs5_agentic_portfolio/
  |-- src/
    |-- services/
      integration/portfolio_data_service.py   # CS1-CS4 integration
      tracking/assessment_history.py          # NEW: History
      analytics/fund_air.py                   # NEW: Fund-AI-R
      observability/metrics.py                # NEW: Prometheus
    |-# NEW: MCP Server
      mcp/server.py
    |-- agents/
      state.py, specialists.py, supervisor.py # LangGraph
    |-- dashboard/
      app.py                                  # Streamlit
      components/evidence_display.py          # NEW: Evidence UI
  |-# NEW: Tests
    tests/test_mcp_integration.py
  +-- README.md
```

"Claude, prepare the IC meeting for NVIDIA."

The agentic PE Org-AI-R platform — powered by YOUR CS1-CS4 work.

MCP + LangGraph + No Mock Data

Designed by Sri Krishnamurthy, CFA · sri@quantuniversity.com

Copyright © 2025 QuantUniversity LLC. All Rights Reserved.

Version 4.0 — All LLM-as-Judge technical review issues resolved